

Multi-Table JOIN:

The Simplest Explanation

Imagine you are building a sandwich. You start with bread, then add cheese, then add tomato, then add lettuce. Each new layer goes on top of everything you already have. You never go back to just the bare bread.

SQL JOINS work the same way.

You start with one table. Then each LEFT JOIN adds a new layer of data on top of what you already built. The next JOIN does NOT go back to the original tables. It works on the combined result of everything so far.

Step	What You Add	What You Have Now
Start	Bread (base)	Bread
+1	Cheese	Bread + Cheese
+2	Tomato	Bread + Cheese + Tomato
+3	Lettuce	Bread + Cheese + Tomato + Lettuce
Result		The Complete Sandwich!

This is exactly how SQL JOINS build the result, step by step.

The Query

```
SELECT first_name, last_name, salary, job_title,  
       department_name, city, country_name, region_name  
FROM hr.employees e  
     LEFT JOIN hr.departments d ON e.department_id = d.department_id  
     LEFT JOIN hr.jobs j ON e.job_id = j.job_id  
     LEFT JOIN hr.locations l ON d.location_id = l.location_id  
     LEFT JOIN hr.countries c ON l.country_id = c.country_id  
     LEFT JOIN hr.regions r ON c.region_id = r.region_id;
```

We use 2 employees this time. Alice has a department, but Kimberly has NULL department_id. This is the key difference.

The Original Tables

hr.employees (alias: e)

employee_id	first_name	job_id	department_id	salary
107	Alice	IT_PROG	60	5000
178	Kimberly	SA_REP	NULL	6000

Kimberly has NULL department_id - she has no department!

hr.departments (alias: d)

department_id	department_name	location_id
60	IT	1500

hr.jobs (alias: j)

job_id	job_title
IT_PROG	Programmer
SA_REP	Sales Rep

hr.locations (alias: l)

location_id	city	country_id
1500	London	UK

hr.countries (alias: c)

country_id	country_name	region_id
UK	United Kingdom	2

hr.regions (alias: r)

region_id	region_name
2	Americas

Step-by-Step: Watch the Result Grow

Step 1: FROM employees

We start with both employees. Alice has department_id = 60. Kimberly has department_id = NULL.

first_name	last_name	salary	job_id	department_id
Alice	Smith	5000	IT_PROG	60
Kimberly	Grant	6000	SA_REP	NULL

Result after Step 1: 2 employees, 5 columns each.



LEFT JOIN departments d ON e.department_id = d.department_id

Step 2: + departments

Alice: department_id = 60 matches! The database finds department "IT" and adds it. Kimberly: department_id = NULL. NULL never matches anything. But because this is LEFT JOIN, Kimberly is NOT removed. She stays in the result with NULL for all department columns.

first_name	last_name	salary	job_id	dept_id	department_name
Alice	Smith	5000	IT_PROG	60	IT
Kimberly	Grant	6000	SA_REP	NULL	NULL

Result after Step 2: Alice got "IT". Kimberly has NULL for department_name (orange cells = NULL values). But she is STILL there!



LEFT JOIN jobs j ON e.job_id = j.job_id

Step 3: + jobs

Both employees have job_ids that match. Alice gets "Programmer", Kimberly gets "Sales Rep". NULL columns do NOT prevent matching other tables! Kimberly had NULL department_id, but her job_id = SA_REP is perfectly valid.

first_name	last_name	salary	job_id	dept_id	dept_name	job_title
Alice	Smith	5000	IT_PROG	60	IT	Programmer
Kimberly	Grant	6000	SA_REP	NULL	NULL	Sales Rep

Result after Step 3: Both get job titles. NULL department does not block job join.



LEFT JOIN locations l ON d.location_id = l.location_id

Step 4: + locations (THE KEY STEP!)

ON d.location_id = l.location_id

Where does "d.location_id" come from? From the result of Step 3, not the original departments table. In Step 2, the departments data (including location_id) was already merged into the result. Alice has d.location_id = 1500 (from the IT department). Kimberly has d.location_id = NULL (she had no department, so no location_id either). Alice matches: gets "London". Kimberly has NULL, so she gets NULL for city.

first_name	last_name	salary	job_id	dept	job_title	city
Alice	Smith	5000	IT_PROG	IT	Programmer	London
Kimberly	Grant	6000	SA_REP	NULL	Sales Rep	NULL

Alice got "London". Kimberly has NULL for city. But LEFT JOIN keeps her!



LEFT JOIN countries c ON l.country_id = c.country_id

Step 5: + countries

Alice has l.country_id = UK (from the location "London"). Gets "United Kingdom". Kimberly has l.country_id = NULL (no location, no country). Gets NULL.

first_name	last_name	salary	job_id	dept	job_title	city	country_name
Alice	Smith	5000	IT_PROG	IT	Programmer	London	United Kingdom
Kimberly	Grant	6000	SA_REP	NULL	Sales Rep	NULL	NULL



LEFT JOIN regions r ON c.region_id = r.region_id

Step 6: + regions

Alice gets "Americas". Kimberly gets NULL. This is the final result with all 6 tables joined.

first_name	last_name	salary	job_id	dept	job_title	city	country	region
Alice	Smith	5000	IT_PROG	IT	Programmer	London	UK	Americas
Kimberly	Grant	6000	SA_REP	NULL	Sales Rep	NULL	NULL	NULL

Final result: 2 rows, data from 6 tables. Orange = NULL. LEFT JOIN kept Kimberly!

The Big Question: LEFT JOIN vs INNER JOIN

Why does Kimberly appear with LEFT JOIN but disappear with INNER JOIN? Let us compare the two types side by side.

INNER JOIN Result (just the departments step)

INNER JOIN only keeps rows where BOTH sides match. Kimberly has department_id = NULL. NULL never equals anything. So there is NO match. Kimberly is removed from the result immediately.

first_name	last_name	salary	job_id	dept_id	department_name
Alice	Smith	5000	IT_PROG	60	IT
Kimberly	Grant	6000	SA_REP	NULL	NULL

With INNER JOIN: Kimberly is GONE (red row). Only Alice remains.

LEFT JOIN Result (same departments step)

LEFT JOIN keeps ALL rows from the left side (the result so far), even if there is no match on the right side. Kimberly has no matching department, so she gets NULL for all department columns. But she stays in the result.

first_name	last_name	salary	job_id	dept_id	department_name
Alice	Smith	5000	IT_PROG	60	IT
Kimberly	Grant	6000	SA_REP	NULL	NULL

With LEFT JOIN: Kimberly is STILL THERE. Orange = NULL, but the row exists.

Side-by-Side Comparison

	INNER JOIN	LEFT JOIN
Alice (has dept)	INCLUDED	INCLUDED
Kimberly (no dept)	REMOVED	KEPT (with NULLs)
Total rows	1	2
NULL values	Not shown - row gone	Shown in orange

Red = row removed. Green = row kept with NULL values.

Why NULL Cascades Through All JOINS

Kimberly had NULL department_id. When we joined departments, she got NULL for all department columns (department_name, location_id). Then in Step 4, we tried to join locations using d.location_id. But for Kimberly, d.location_id is already NULL. So NULL cascades forward like a chain reaction:

JOIN Step	ON Clause	Kimberly's Value	Result
+ departments	e.department_id = d.department_id	NULL = ?	NULL (no match)
+ jobs	e.job_id = j.job_id	SA_REP = SA_REP	Sales Rep (match!)
+ locations	d.location_id = l.location_id	NULL = ?	NULL (cascaded)
+ countries	l.country_id = c.country_id	NULL = ?	NULL (cascaded)
+ regions	c.region_id = r.region_id	NULL = ?	NULL (cascaded)

Orange rows = NULL cascaded. Green row = successful match (job_id was not NULL!).

The important thing to notice: one NULL can cascade through ALL remaining JOINS. Kimberly had NULL only for department_id. But because of that, she also got NULL for location, country, and region. This is called "NULL propagation" - once a value is NULL, every JOIN that depends on it will also produce NULL.

But her job_id was NOT NULL, so the jobs JOIN still worked! Kimberly got "Sales Rep" as her job title. NULL in one column does NOT affect unrelated columns.

Summary

1. LEFT JOIN keeps all rows from the left side. If there is no match on the right, it fills in NULL values. The row is NOT removed. This is why Kimberly appears in the result even though she has no department.
2. INNER JOIN removes rows with no match. If Kimberly's department_id is NULL, INNER JOIN cannot find a matching department. Kimberly is removed from the result immediately. She never reaches the later JOINS.
3. NULL cascades through the chain. When one JOIN produces NULL, the next JOIN that depends on that NULL value will also produce NULL. Kimberly's NULL department_id caused NULL for location, country, and region. But unrelated columns (like job_title) can still match normally.
4. Each JOIN works on the previous result. In Step 4, d.location_id comes from the result built in Steps 1-3, not from the original departments table. The departments data was already merged in Step 2 and has been traveling with the result ever since.

LEFT JOIN = "Keep everyone, fill in NULL if no match"
INNER JOIN = "Only keep rows that match on both sides"